以下为AI生成的图文笔记的内容

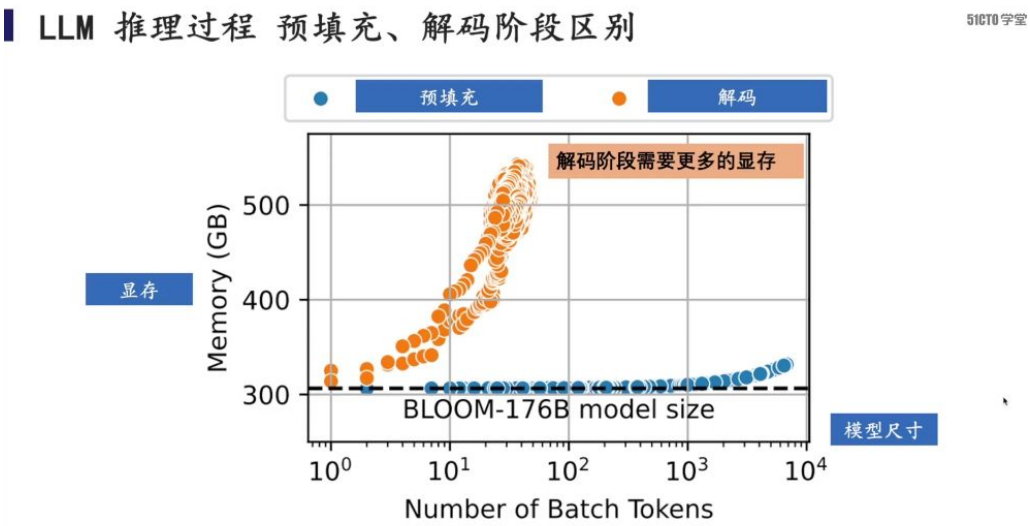
一、大模型推理过程与瓶颈分析总结 **00:08**

1. 推理过程 00:25



- **两个阶段**：大模型推理分为预填充（Prefill）和解码（Decode）两个阶段
- **关键指标**：
  - 首词生成时间（TTFT）：预填充阶段生成第一个token所需时间
  - 每输出词时间（TPOT）：解码阶段每个token生成的时间

1）预填充和解码阶段区别 01:09

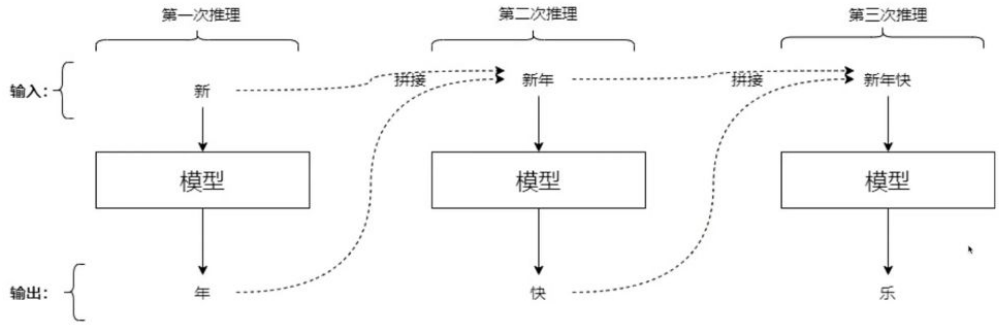


- **显存需求**：解码阶段需要更多显存，预填充阶段需要更多算力
- **吞吐量差异**：预填充阶段吞吐量随batch size增加而提升，解码阶段吞吐量相对稳定
- **计算特点**：
  - 预填充：一次性处理所有输入token
  - 解码：自回归生成，每个新token依赖之前所有token

2）KV cache执行过程 01:34

LLM 推理过程 预填充、解码阶段-KV cache执行过程

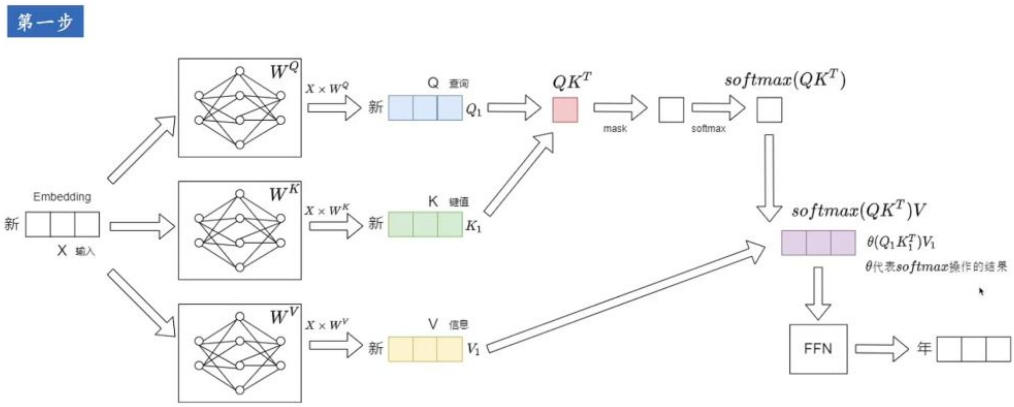
51CTO学堂



-
- **自回归特性**：每个新token生成都依赖之前所有token的计算结果
- **冗余计算**：
  - 嵌入层重复计算
  - K/V向量生成冗余
  - QK^T计算重复
  - softmax操作重复
- **缓存原理**：历史token的K/V向量不变，可复用缓存

LLM 推理过程 预填充、解码阶段-KV cache执行过程

51CTO学堂



-
- **缓存增长**：随着序列长度增加，KV cache会线性增长
- **计算优化**：只需计算新token的Q向量，与缓存的K/V进行注意力计算

2. KV Cache的内存占用 04:57

KV LLM大语言模型的内存占用

51CTO学堂

GPU存储=大模型自身占用内存+KV Cache内存+其他内存

变量	描述	例子
n	当前序列的长度（即已缓存的令牌数量）	1000
l	模型中的层数（Number of layers）	32
b	批处理大小（Batch size）	1
h	每层注意力头的数量（Number of attention heads）	32
dk	每个注意力头的维度（Head dimension）	128
P	每个元素的精度（以字节计）。例如，float16 为 2 字节。	2
2	因子 2，表示每个令牌需要存储 *K* 和 *V* 两组向量。	2

Size (bytes)=2·32·1·32·1000·128·2
Size=524,288,000 字节
大约 500 MB 内存占用
对于一个相对较小的序列长度（1000 个令牌）和单批次，KV Cache 也占用了大量的 GPU 内存（约 500 MB）。将序列长度 *n* 增加到 128,000（长上下文 LLM 的场景）或将批处理大小 *b* 增加到 32，所需的内存将呈线性爆炸式增长。

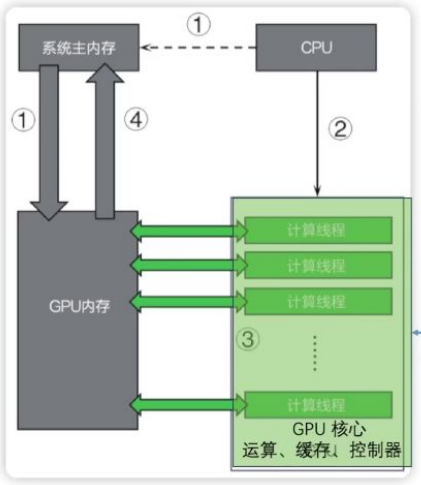
- **内存组成**：GPU存储=模型参数+KV Cache+其他内存
- **计算公式**：
  - 序列长度n=1000时约占用500MB
  - 内存=2×n×l×b×h×d_k×精度
- **关键参数**：
  - n：序列长度（已缓存token数）
  - l：模型层数（如32层）
  - b：batch size
  - h：注意力头数
  - d_k：每个注意力头维度

3. 硬件瓶颈和性能约束 05:26

1）内存中的数据如何交给GPU进行运算 05:28

硬件瓶颈和性能约束 内存中的数据如何交给GPU进行运算？

51CTO学堂



1. CPU发起DMA（Direct Memory Access，直接内存访问），将系统主内存中的数据复制到GPU内存中。加载模型、数据集。
2. CPU向GPU注入指令。推理、训练。
3. GPU中的多个计算线程并行执行CPU指令。
4. GPU发起DMA，将GPU内存中的数据复制到系统主内存中。推理、训练的结果写回主内存。

```
// 核函数（Kernel Function）定义
__global__ void VecAdd(float* A, float* B, float* C)
{
  int i = threadIdx.x; /* 根据当前线程取向量元素 */
  C[i] = A[i] + B[i];  /* 调用CUDA Core执行加法指令 */
}

int main(int argc, char** argv)
{
  ...
  // 调用核函数，启动N个线程
  VecAdd<<<1, N>>>(A, B, C);
  ...
}
```

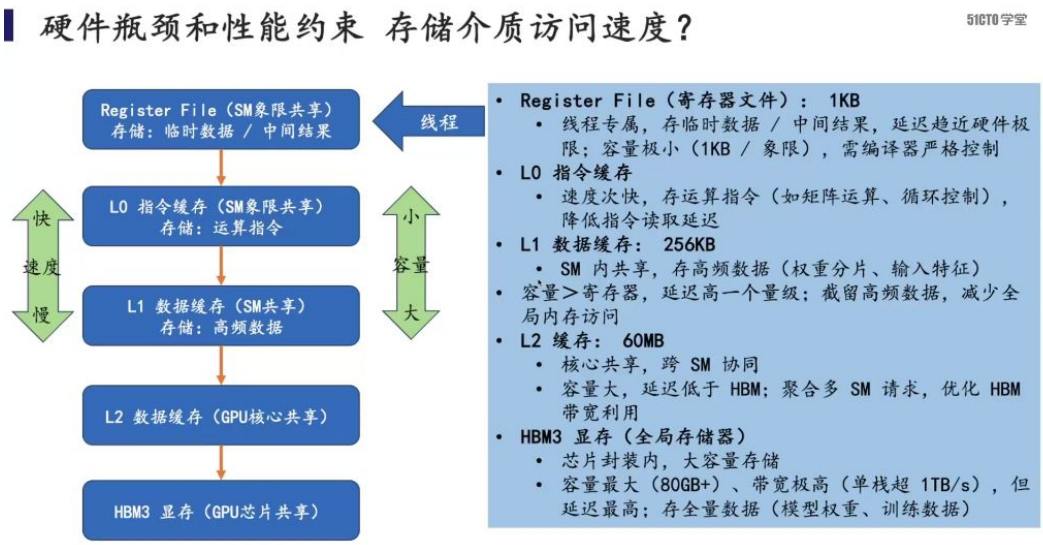
向量A和B求和

启动N个线程计算

- **数据传输流程**：
  - CPU通过DMA将数据从主存复制到GPU显存
  - CPU向GPU注入指令
  - GPU并行执行计算
  - 结果通过DMA写回主存

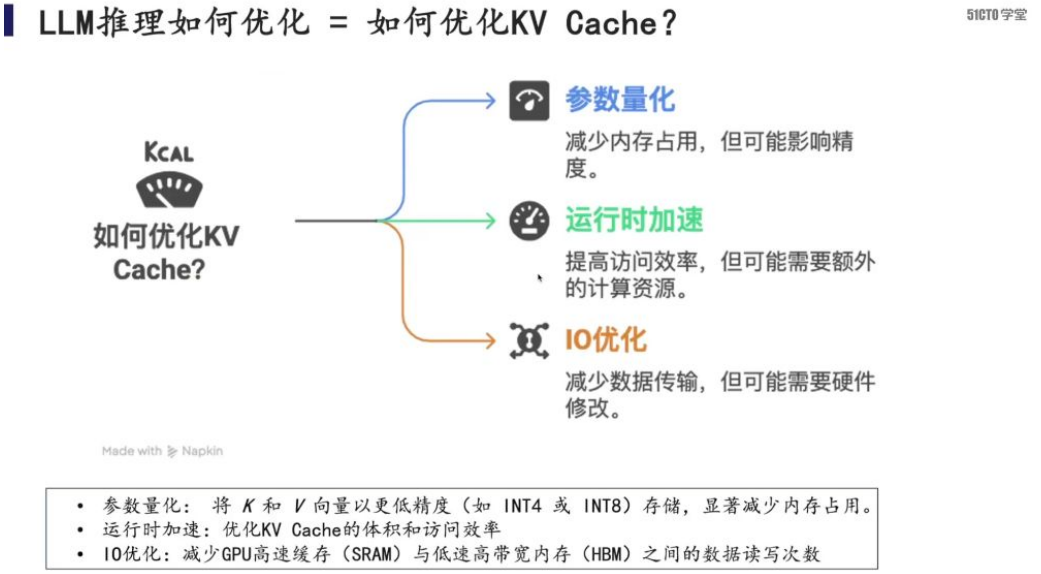
- **硬件架构**:
  - HBM显存：带宽800GB/s
  - PCI-E 5.0：双向带宽64GB/s
  - NVLink：18通道互联

2）存储介质访问速度 06:56



- **寄存器文件**：1KB/线程，延迟最低
- **L0指令缓存**：存储运算指令
- **L1数据缓存**：256KB/SM，存高频数据
- **L2缓存**：60MB，核心共享
- **HBM显存**：80GB+容量，TB/s级带宽但延迟最高

4. LLM推理如何优化 07:14



- **参数量化**：将K/V向量用INT4/INT8存储
- **运行时加速**：优化KV Cache体积和访问效率
- **IO优化**：减少SRAM与HBM间数据交换
- **关键权衡**:
  - 量化会损失精度
  - 加速可能需要额外计算资源

- IO优化可能需要硬件修改

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
大模型推理阶段划分	分为预填充阶段（记录问题并生成首个token）和解码阶段（逐token生成输出）	**预填充阶段侧重算力，解码阶段侧重显存**	★★
KV Cache机制	缓存历史生成的K/V向量以避免重复计算，**用空间换时间**	KV Cache会随输入/输出序列增长而**持续膨胀**	★★★
Transformer架构瓶颈	注意力计算中的Q/K/V向量存在冗余（尤其是K/V），需通过缓存复用	**K/V向量复用**是优化关键，但需权衡内存占用	★★★★
硬件资源占用分析	模型参数内存公式：涉及序列长度、层数、批量大小、多头数等	**显存存参数**，**GPU核心（SM）负责运算**	★★★
GPU核心结构	SM包含多种精度运算单元（FP32/INT32等）及缓存层级	**缓存离SM越近速度越快容量越小**	★★
优化三大方向	1. **参数量化**降低内存占用 2. **加速KV Cache访问** 3. **IO优化**减少显存与SM间数据搬运	**KV Cache体积控制**与**访问效率**是核心矛盾	★★★★